

VITyarthi - Build Your Own Project

ACADEMIC MANAGER

Personal Student Task Scheduler

Project Report

Subject: Introduction to Programming and Problem Solving

Submitted by: Mouli Goyal

Registration no. – 26BAI10684

Fall semester 2026–27

Introduction

Academic Manager is a simple Python command-line project made for students as it helps them keep a track of their classes, assignments, available study hours, and assignment progress in one place.

The project was made for the VITyarthi Build Your Own Project activity for Introduction to Programming and Problem Solving. This program uses basic Python concepts taught in the course, such as lists, dictionaries, tuples, functions, loops, conditions, input, output, and modules.

Problem Statement

Students have to constantly manage classes, assignments, deadlines, and study time during the semester. Keeping these details separately can make it harder to remember pending work and available study time, which makes the student less efficient.

Academic Manager provides a small command-line solution. The user can enter classes and assignments, add available study hours, generate a simple study plan, and mark assignments as completed.

Objectives

- To provide a simple way to store and view a student's classes, assignments and their priorities.
- To let the user enter available study hours.
- To generate a basic study plan from the entered study hours.
- To track completed and pending assignments.
- To practice basic Python programming concepts through a useful project.

Scope of the Project

The project is designed for basic academic task management. It is a command-line application and does not use a database or graphical interface.

- Manage a personal timetable, assignments and priorities.
- Enter available study hours.
- Generate a simple study plan.
- Mark assignments as completed.
- View basic progress

Functional Requirements

Module	Function	Input/Output
Timetable	Add and view classes	Input: day, subject, time. Output: class list.
Assignments	Add and view assignments	Input: name, subject, deadline, priority. Output: assignment list.
Scheduler	Add study hours and generate a plan	Input: day and time. Output: study plan.
Progress	Mark assignments complete and view progress	Input: assignment number. Output: completed/pending

Non-Functional Requirements

It includes more than the required four non-functional requirements. The instructions list areas such as performance, usability, reliability, maintainability, error handling, and resource efficiency.

The system should:

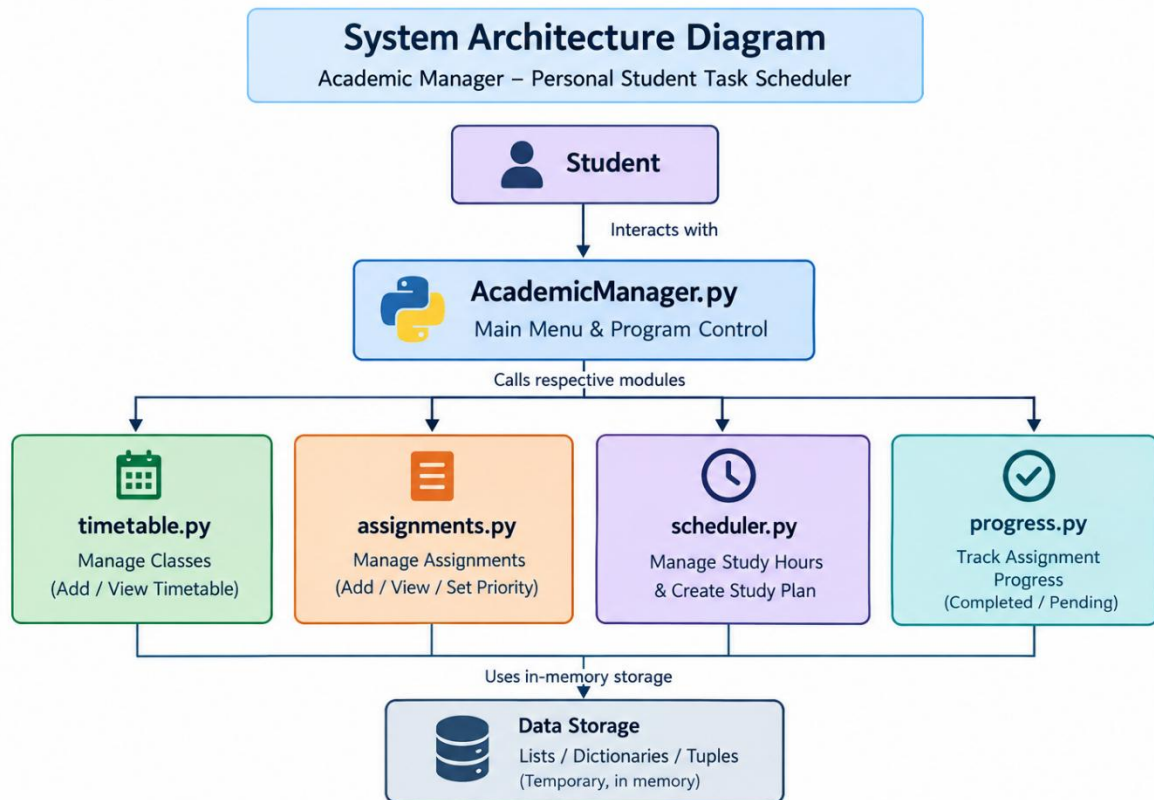
- Be simple and easy to use.
- Give clear messages for user actions and invalid choices.
- Respond quickly to user input.
- Use a modular and organized Python structure.
- Use basic Python concepts covered in the course.
- Be easy to maintain and modify.

System Architecture

File	Purpose
AcademicManager.py	Main menu and program control
timetable.py	Adds and displays classes
assignments.py	Adds and displays assignments
scheduler.py	Adds study hours and creates a study plan
progress.py	Marks assignments complete and shows progress

The application uses a simple modular structure. AcademicManager.py controls the main menu and calls functions from the other modules.

Data is stored in lists while the program is running. Dictionaries store classes and assignment details. Tuples store a study day and study time. No permanent storage is used.



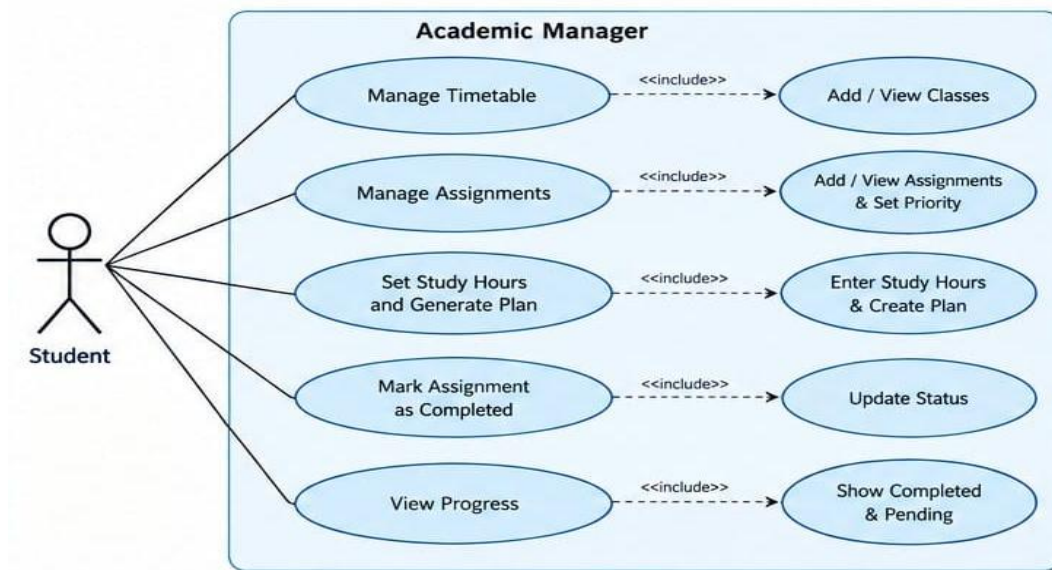
Design Diagrams

The VITYarthi report requirements ask for a use case diagram, workflow diagram, sequence diagram, and class/component diagram. An ER diagram is required when storage is used.

The diagrams below describe the actual project structure. Since the project does not use a database, an ER diagram is not needed.

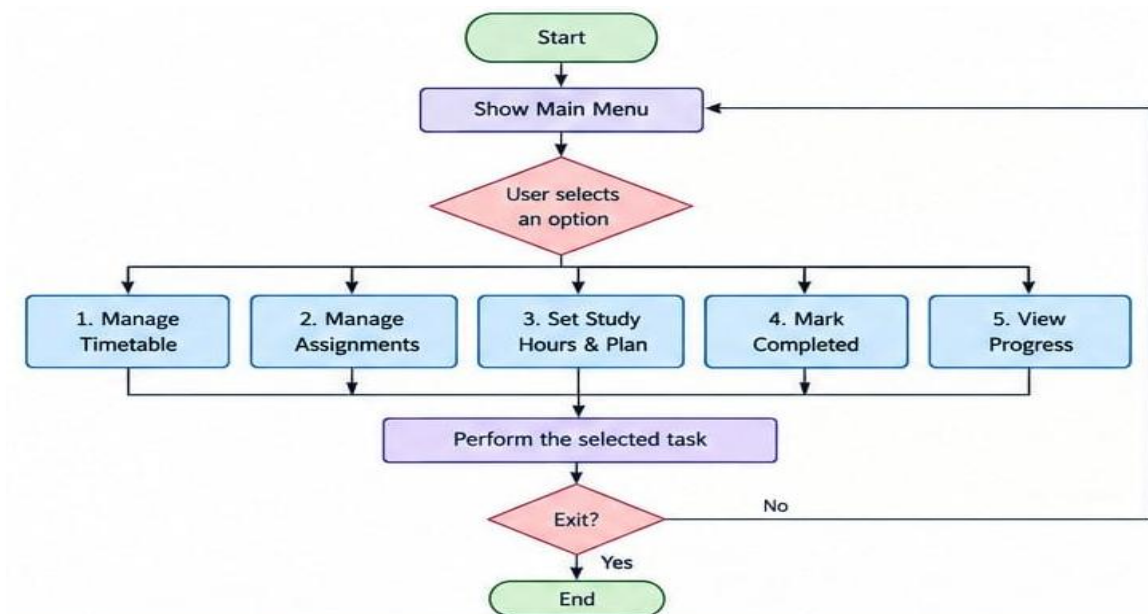
1. Use Case Diagram

- Student Add/View Classes - Add/View Assignments, Add Study Hours, Generate Study Plan, Mark Assignment Complete, View Progress.
- Main actor: Student.
- The student interacts with all five project modules through the main menu



Workflow Diagram

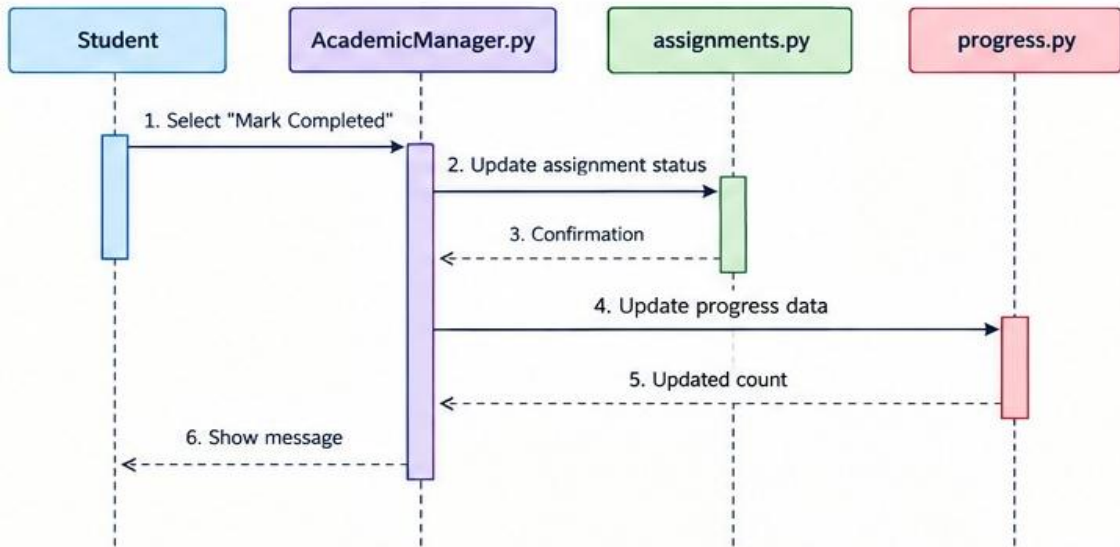
- Start > Main Menu > Select Option > Enter Data / Perform Action > Display Result > Return to Main > Menu > Exit
- For study planning: Add Study Hours > Generate Study Plan > Display Assignment and Study Time.



Sequence Diagram

- Student selects an option in AcademicManager.py.
- AcademicManager.py calls the required module function.
- The module takes input and updates the related list.

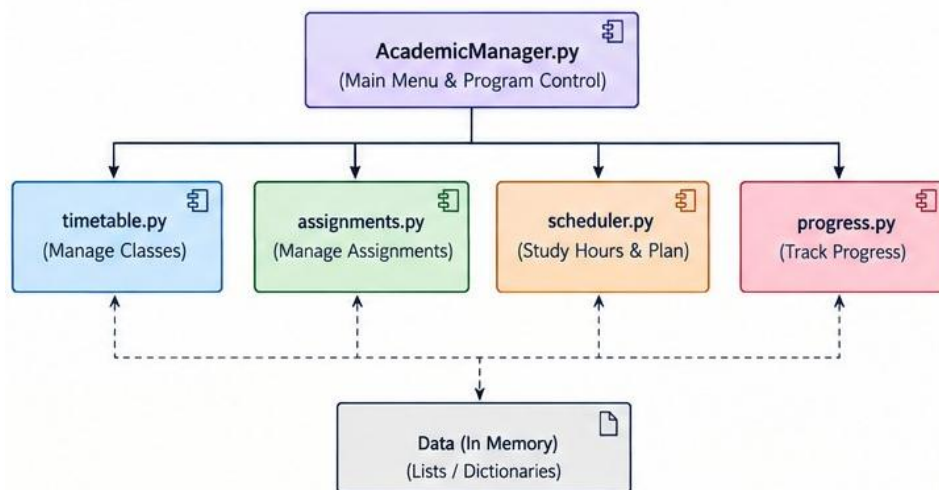
- The module prints the result.
- Control returns to the main menu.



Component Diagram

Student > AcademicManager.py > {timetable.py, assignments.py, scheduler.py, progress.py)

scheduler.py also reads assignment data from assignments.py when generating the study plan. progress.py receives the assignment list from AcademicManager.py.



Storage Design

No database is used. The application stores data only while it is running.

- timetable: list of dictionaries containing day, subject, and time.
- assignments: list of dictionaries containing name, subject, deadline, priority, and completed status.
- Study hours: list of tuples containing day and available time.

Design Decisions and Rationale

- Python was selected because it is the language used in the course and is suitable for a small command-line project.
- Separate modules were used so each file has one main responsibility.
- Lists were used for collections of classes, assignments, and study hours.
- Dictionaries were used because assignment and class details have named fields.
- Tuples were used to store a study day and time together.
- A command-line interface was used to keep the project focused on programming fundamentals.
- The project does not use a database because the current scope is a basic beginner project.
- These choices keep the implementation close to the concepts covered in the subject.

Implementation Details

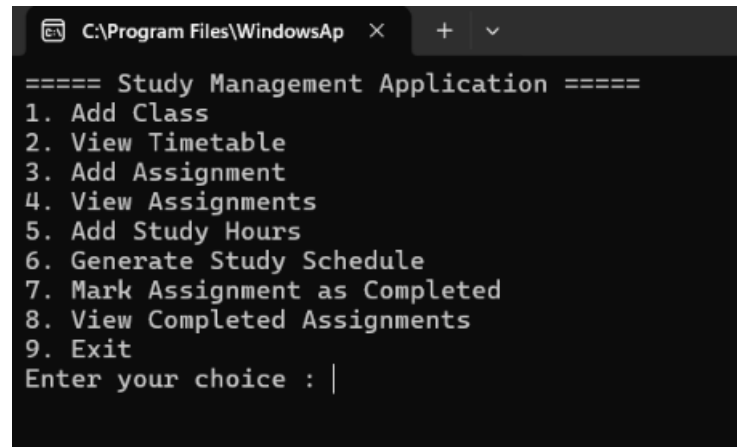
The program starts from AcademicManager.py. It displays a menu and calls the correct function based on the user's choice.

The main implementation uses input(), print(), if/elif/else, for loops, functions, lists, dictionaries, tuples, append(), and Python modules.

- Timetable: add class() stores a class dictionary in the timetable list. view timetable() displays the stored classes.
- Assignments: add assignment() collects assignment details and stores them in the assignments list. view assignments() displays them.
- Scheduler: add studyhours() stores a (day, time) tuple. generate plan() uses the available study hours for pending assignments.
- Progress: mark complete() changes the selected assignment's completed value to True. View progress() counts completed and pending assignments.

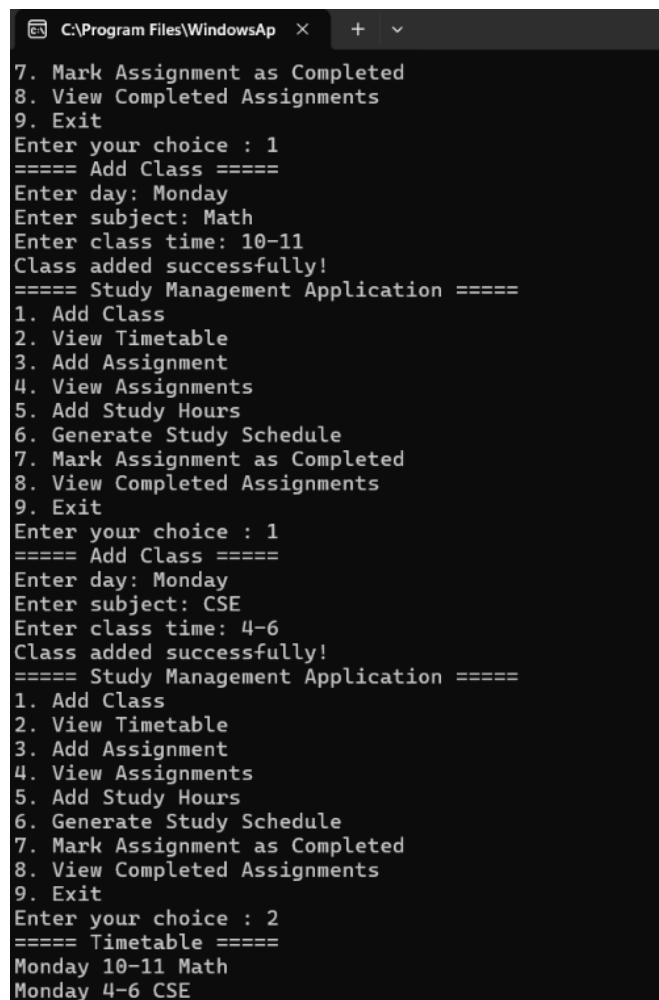
Basic validation is included, such as checking whether lists are empty and whether a selected assignment number is within the valid range.

Results

A screenshot of a Windows command prompt window titled 'C:\Program Files\WindowsAp'. The application displays a menu titled '==== Study Management Application =====' with the following options: 1. Add Class, 2. View Timetable, 3. Add Assignment, 4. View Assignments, 5. Add Study Hours, 6. Generate Study Schedule, 7. Mark Assignment as Completed, 8. View Completed Assignments, and 9. Exit. The prompt 'Enter your choice : |' is shown at the bottom.

```
C:\Program Files\WindowsAp >
==== Study Management Application =====
1. Add Class
2. View Timetable
3. Add Assignment
4. View Assignments
5. Add Study Hours
6. Generate Study Schedule
7. Mark Assignment as Completed
8. View Completed Assignments
9. Exit
Enter your choice : |
```

Main menu

A screenshot of a Windows command prompt window titled 'C:\Program Files\WindowsAp'. It shows the user selecting option 1 (Add Class) from the main menu. The application prompts for 'day', 'subject', and 'class time'. The first class added is 'Monday', 'Math', '10-11'. The second class added is 'Monday', 'CSE', '4-6'. Both are confirmed with 'Class added successfully!'. The main menu is displayed again, and the user selects option 2 (View Timetable). The application then displays the created timetable.

```
C:\Program Files\WindowsAp >
7. Mark Assignment as Completed
8. View Completed Assignments
9. Exit
Enter your choice : 1
==== Add Class ====
Enter day: Monday
Enter subject: Math
Enter class time: 10-11
Class added successfully!
==== Study Management Application =====
1. Add Class
2. View Timetable
3. Add Assignment
4. View Assignments
5. Add Study Hours
6. Generate Study Schedule
7. Mark Assignment as Completed
8. View Completed Assignments
9. Exit
Enter your choice : 1
==== Add Class ====
Enter day: Monday
Enter subject: CSE
Enter class time: 4-6
Class added successfully!
==== Study Management Application =====
1. Add Class
2. View Timetable
3. Add Assignment
4. View Assignments
5. Add Study Hours
6. Generate Study Schedule
7. Mark Assignment as Completed
8. View Completed Assignments
9. Exit
Enter your choice : 2
==== Timetable ====
Monday 10-11 Math
Monday 4-6 CSE
```

Timetable created by adding classes

```

===== Study Management Application =====
1. Add Class
2. View Timetable
3. Add Assignment
4. View Assignments
5. Add Study Hours
6. Generate Study Schedule
7. Mark Assignment as Completed
8. View Completed Assignments
9. Exit
Enter your choice : 3
===== Add Assignment =====
Enter assignment name: PYTHON
Enter subject: CSE
Enter deadline: 30/9/2026
1. High
2. Medium
3. Low
Enter priority : 1
Assignment added successfully!
===== Study Management Application =====
1. Add Class
2. View Timetable
3. Add Assignment
4. View Assignments
5. Add Study Hours
6. Generate Study Schedule
7. Mark Assignment as Completed
8. View Completed Assignments
9. Exit
Enter your choice : 3
===== Add Assignment =====
Enter assignment name: Calculus
Enter subject: Math
Enter deadline: 5/10/2026
1. High
2. Medium
3. Low
Enter priority : 3
Assignment added successfully!

```

```

===== Study Management Application =====
1. Add Class
2. View Timetable
3. Add Assignment
4. View Assignments
5. Add Study Hours
6. Generate Study Schedule
7. Mark Assignment as Completed
8. View Completed Assignments
9. Exit
Enter your choice : 4
===== Assignments =====
Name: PYTHON
Subject: CSE
Deadline: 30/9/2026
Priority: High
Status: Pending

Name: Calculus
Subject: Math
Deadline: 5/10/2026
Priority: Low
Status: Pending

```

assignments, priorities, and status

```

===== Study Management Application =====
1. Add Class
2. View Timetable
3. Add Assignment
4. View Assignments
5. Add Study Hours
6. Generate Study Schedule
7. Mark Assignment as Completed
8. View Completed Assignments
9. Exit
Enter your choice : 5
===== Add Study Hours =====
Enter day: Monday
Enter available time: 2-3
Study hours added successfully!

```

```

===== Study Management Application =====
1. Add Class
2. View Timetable
3. Add Assignment
4. View Assignments
5. Add Study Hours
6. Generate Study Schedule
7. Mark Assignment as Completed
8. View Completed Assignments
9. Exit
Enter your choice : 6
===== Study Plan =====
Assignment: PYTHON
Study on: Monday
Study time: 2-3

```

study plan

```

===== Study Management Application =====
1. Add Class
2. View Timetable
3. Add Assignment
4. View Assignments
5. Add Study Hours
6. Generate Study Schedule
7. Mark Assignment as Completed
8. View Completed Assignments
9. Exit
Enter your choice : 7
1 PYTHON
2 Calculus
Enter number: 2
Completed!
===== Study Management Application =====
1. Add Class
2. View Timetable
3. Add Assignment
4. View Assignments
5. Add Study Hours
6. Generate Study Schedule
7. Mark Assignment as Completed
8. View Completed Assignments
9. Exit
Enter your choice : 8
Total: 2
Completed: 1
Pending: 1

```

Progress of completed and pending assignment

```

===== Study Management Application =====
1. Add Class
2. View Timetable
3. Add Assignment
4. View Assignments
5. Add Study Hours
6. Generate Study Schedule
7. Mark Assignment as Completed
8. View Completed Assignments
9. Exit
Enter your choice : 9
Exiting the application. Goodbye!

```

Testing Approach

Testing was done by running the program and checking the main functions with normal and invalid inputs.

Test	Input / Action	Expected Result
Add class	Enter day, subject and time	Class is added and shown in timetable
Add assignment	Enter details and priority	Assignment is added with selected priority
Add study hours	Enter day and time	Study hour is stored
Generate plan	Select study plan after adding data	Pending assignments are assigned to entered study hours.
Complete assignment	Enter a valid assignment number	Assignment status changes to Completed.
Progress	View progress	Total, completed, and pending counts are shown.
Invalid menu choice	Enter an option outside the menu	Program shows an invalid-choice message.

Challenges Faced

- Connecting multiple Python files through imports and function calls.
- Keeping assignment data available to the scheduler and progress module.
- Handling user choices correctly in the menu.
- Checking empty lists before displaying or processing data.
- Keeping the project simple while dividing it into separate modules.

Learnings and Key Takeaways

- Learned how to divide a Python program into modules.
- Practiced lists, dictionaries, tuples, functions, loops, and conditions.
- Learned how to pass a list from one module to a function in another module.
- Practiced basic input validation.
- Learned how to test a command-line program.
- Learned how project documentation and source code are connected.

GitHub and Version Control

The project is organized as a GitHub repository. The repository contains the Python source files and project documentation.

- README.md – project overview, features, technologies, run instructions, and testing.
- statement.md – problem statement, scope, target users, and high-level features.
- requirements.md – functional and non-functional requirements.
- design.md – project design, modules, workflow, and data structures.
- testing.md – basic testing record.
- Python source files – complete project implementation.

Git is used for version control so changes to the project can be tracked.

Limitations

- Data is lost when the program is closed.
- The study plan is simple and does not automatically compare deadlines or priorities.
- There is no login system.
- There is no graphical user interface.
- There is no database.

Future Enhancements

- Add permanent data storage.
- Use deadline and priority while generating the study plan.
- Add weekly progress reports.
- Add reminders for pending assignments.
- Add a graphical user interface.
- Add better input validation for dates and times.

Conclusion

- Academic Manager is a small Python project for managing basic academic tasks. It combines timetable management, assignment management, study planning, and progress tracking in one command-line program.
- The project uses the programming concepts covered in Introduction to Programming and Problem Solving. Its modular structure makes the code easier to read and change. The project also includes requirements, design documentation, testing, and GitHub files needed for submission

References

- VITyarthi – Build Your Own Project: General Project Instructions & Submission Guidelines.
- Python documentation and course material used for basic Python syntax and modules.
- Project source code and project documentation prepared for this submission.